STAR RX72N - rx_session Library

1.0.0

Generated by Doxygen 1.16.1

Chapter 1

Directory Hierarchy

1.1 Directories

Chapter 2

File Index

2.1 File List

Here is a list of all files with brief descriptions:

Chapter 3

Directory Documentation

3.1 libs/rx_session/inc Directory Reference

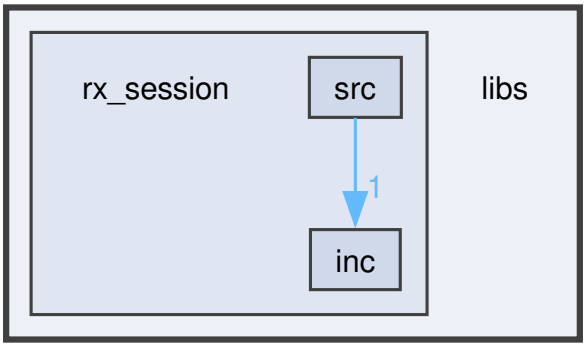
Directory dependency graph for inc:

Files

- file rx_session.h
  Shared session state for cross-transport sequence continuity.

3.2 libs Directory Reference

Directory dependency graph for libs:

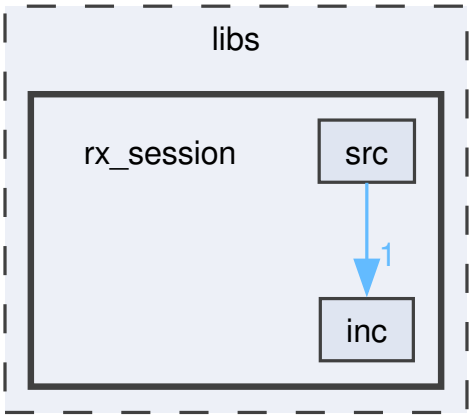


Directories

- directory rx_session

3.3 libs/rx_session Directory Reference

Directory dependency graph for rx_session:

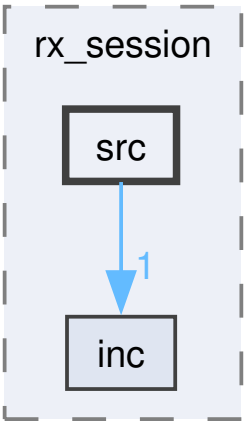


Directories

- directory inc
- directory src

3.4 libs/rx_session/src Directory Reference

Directory dependency graph for src:



Files

- file rx_session.c

  Shared session state implementation for cross-transport sequence continuity.

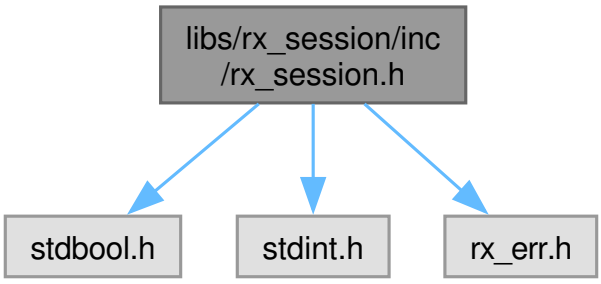
Chapter 4

File Documentation

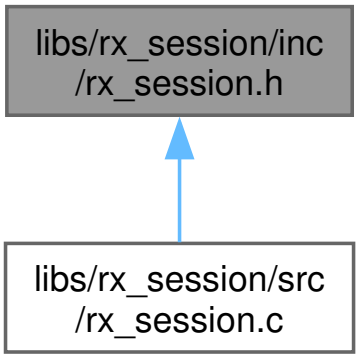
4.1 libs/rx_session/inc/rx_session.h File Reference

Shared session state for cross-transport sequence continuity.

#include <stdbool.h>
#include <stdint.h>
#include "rx_err.h"
Include dependency graph for rx_session.h:



This graph shows which files directly or indirectly include this file:



Data Structures

- struct rx_session_state_t

  Shared session state for cross-transport sequence continuity.

Enumerations

- enum rx_session_constants_t : uint16_t { k_session_initial_sequence = 0 , k_session_diff_exact_match = 0 , k_session_max_gap_tolerance = 32768 , k_session_seq_wrap_mask = 0xFFFF }

  Session state configuration constants.
- enum rx_session_validate_result_t : uint8_t { k_session_validate_ok = 0 , k_session_validate_gap = 1 , k_session_validate_fail = 2 }

  Result codes for sequence validation.

Functions

- rx_err_t rx_session_init (rx_session_state_t *state)

  Initialize session state with zeroed sequences.
- rx_err_t rx_session_deinit (rx_session_state_t *state)

  Deinitialize session state and release resources.
- rx_err_t rx_session_next_tx (rx_session_state_t *state, uint16_t *sequence)

  Get next TX sequence number and increment the counter.
- rx_err_t rx_session_validate_rx (rx_session_state_t *state, uint16_t received_seq, rx_session_validate_result_ *result)

  Validate a received sequence number.
- rx_err_t rx_session_reset (rx_session_state_t *state)

  Reset both TX and RX sequences to zero.
- rx_err_t rx_session_get_tx (const rx_session_state_t *state, uint16_t *sequence)

  Get current TX sequence without incrementing.
- rx_err_t rx_session_get_rx (const rx_session_state_t *state, uint16_t *sequence)

  Get current expected RX sequence.

4.1.1 Detailed Description

Shared session state for cross-transport sequence continuity.

Provides a single, shared session state (TX and RX sequence counters) that is used by BOTH USB CDC and SPI transport layers. When the active transport switches (e.g., USB at seq=105 to SPI), SPI continues at seq=106 because both transports reference the same session state.

This module mirrors the Go gateway's manager/session.go implementation:

- NextTxSequence() -> rx_session_next_tx()
- ValidateRxSequence() -> rx_session_validate_rx()
- Reset() -> rx_session_reset()

4.1.1.1 Architecture

```
rx_comm_manager_t (owns rx_session_state_t)
    |
    +-- rx_usb_comm_handle_t.session --> &session_state
    |
    +-- rx_spi_comm_handle_t.session --> &session_state
```

4.1.1.2 Thread Safety

All public functions are thread-safe. Internal state is protected by a ThreadX mutex, allowing safe concurrent access from the comm_task thread, USB ISR callbacks, and SPI ISR callbacks.

4.1.1.3 Gap Tolerance (Matching Go Gateway)

The RX sequence validator accepts small gaps (up to 10 frames) to handle occasional packet loss on the lightweight USB CDC transport. This matches the Go gateway's MaxGapTolerance = 10 behavior:

Condition	Action
diff == 0	Accept, increment rx_sequence
0 < diff < 10	Accept with warning, update rx_sequence
diff >= 10	Reject, log error

NASA Power of 10 Compliance:

- Rule 1: No goto/setjmp/longjmp/recursion
- Rule 2: Fixed loop bounds (no loops in this module)
- Rule 3: No dynamic allocation (caller provides state struct)
- Rule 5: Precondition/postcondition validation on all functions
- Rule 7: All return values checked
- Rule 8: Macros only for conditional compilation
- Rule 10: Compiled with -Wall -Wextra -Werror

See also

star-gateway/internal/manager/session.go Go reference implementation

rx_usb_comm.h USB CDC transport (uses session for sequence tracking)

rx_spi_comm.h SPI transport (uses session for sequence tracking)

rx_comm_manager.h Communication manager (owns session instance)

Since

Version 1.0.0

Locked, Inc.

February 2026

Definition in file rx_session.h.

4.1.2 Data Structure Documentation

4.1.2.1 struct rx_session_state_t

Shared session state for cross-transport sequence continuity.

Holds TX and RX sequence counters shared between USB CDC and SPI transports. The rx_comm_manager_t owns a single instance and passes a pointer to both transport handles during initialization.

4.1.2.2 Memory Layout (8 bytes, no padding)

Offset	Size	Field	Description
0x00	2 B	tx_sequence	Next TX sequence to assign
0x02	2 B	rx_sequence	Next expected RX sequence
0x04	1 B	initialized	Initialization flag
0x05	3 B	(padding)	Alignment padding

Invariant

tx_sequence and rx_sequence are only modified under mutex protection

initialized is true after successful rx_session_init()

Example:

```
// Owned by comm manager, shared with both transports
static rx_session_state_t g_session;

rx_err_t err = rx_session_init(&g_session);
if (err != k_rx_ok) {
  rx_log_error("SESSION", "Init failed");
  return err;
}

// Pass to both transport handles
usb_handle.session = &g_session;
spi_handle.session = &g_session;
```

Note

Do not access fields directly. Use the rx_session_*() API functions.

Warning

This struct must be statically allocated (no malloc).

See also

rx_session_init() Initialize before use

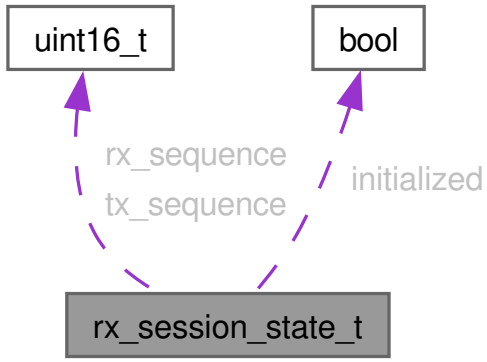
rx_session_deinit() Cleanup when done

Since

Version 1.0.0

Definition at line 208 of file rx_session.h.

Collaboration diagram for rx_session_state_t:



Data Fields

bool	initialized	True after successful rx_session_init()
uint16_t	rx_sequence	Next expected RX sequence number (wraps at 65535)
uint16_t	tx_sequence	Next TX sequence number to assign (wraps at 65535)

4.1.3 Enumeration Type Documentation

4.1.3.1 rx_session_constants_t

enum rx_session_constants_t : uint16_t

Session state configuration constants.

Configuration parameters for the session state module. These match the Go gateway's constants in manager/session.go.

See also

star-gateway/internal/manager/session.go Go reference values

Since

Version 1.0.0

Enumerator

k_session_initial_sequence	Initial TX/RX sequence value after init or reset. Both TX and RX sequence counters start at this value. Used by rx_session_init() and rx_session_reset().
k_session_diff_exact_match	Exact-match diff value for sequence validation. When the difference between received and expected sequence is zero, the sequence matches exactly (most common case).
k_session_max_gap_tolerance	Maximum allowable gap between expected and received sequence numbers. When the received sequence is ahead of expected by this amount or less, the frame is accepted and the RX sequence catches up. Gaps larger than this are rejected as transport failures. Set to half the 16-bit wrap window so any "ahead" sequence is accepted (only true past-window duplicates are rejected). This relaxed bound lets ad-hoc bring-up tools (cmd_debug, vel_test, grpcurl one-shots) drive the link without re-implementing the full session library. The production gateway tracks sequences precisely via rx_session and stays well under this limit, so the relaxation is invisible to it.
k_session_seq_wrap_mask	Sequence number wraparound mask (16-bit unsigned). Applied after incrementing sequence counters to ensure they wrap around at 65535 back to 0. Matches Go: (seq + 1) & 0xFFFF

Definition at line 96 of file rx_session.h.

```
00096         : uint16_t {
00097     /**
00098      * @brief Initial TX/RX sequence value after init or reset
00099      *
00100      * @details
00101      * Both TX and RX sequence counters start at this value. Used by
00102      * rx_session_init() and rx_session_reset().
00103      */
00104     k_session_initial_sequence = 0,
00105
00106     /**
00107      * @brief Exact-match diff value for sequence validation
00108      *
00109      * @details
00110      * When the difference between received and expected sequence is zero,
00111      * the sequence matches exactly (most common case).
00112      */
00113     k_session_diff_exact_match = 0,
00114
00115     /**
00116      * @brief Maximum allowable gap between expected and received sequence numbers
00117      *
00118      * @details
00119      * When the received sequence is ahead of expected by this amount or less,
00120      * the frame is accepted and the RX sequence catches up. Gaps larger than
00121      * this are rejected as transport failures.
00122      *
00123      * Set to half the 16-bit wrap window so any "ahead" sequence is accepted
00124      * (only true past-window duplicates are rejected). This relaxed bound
00125      * lets ad-hoc bring-up tools (cmd_debug, vel_test, grpcurl one-shots)
00126      * drive the link without re-implementing the full session library.
00127      * The production gateway tracks sequences precisely via rx_session and
00128      * stays well under this limit, so the relaxation is invisible to it.
00129      */
00130     k_session_max_gap_tolerance = 32768,
00131
00132     /**
00133      * @brief Sequence number wraparound mask (16-bit unsigned)
00134      *
00135      * @details
00136      * Applied after incrementing sequence counters to ensure they wrap
00137      * around at 65535 back to 0. Matches Go: `(seq + 1) & 0xFFFF`
00138      */
```

```
00139   k_session_seq_wrap_mask = 0xFFFF,
00140 } rx_session_constants_t;
```

4.1.3.2 rx_session_validate_result_t

enum rx_session_validate_result_t : uint8_t

Result codes for sequence validation.

Returned by rx_session_validate_rx() to indicate the validation outcome. This provides more information than a simple bool, allowing callers to distinguish between exact matches, gap acceptance, and rejection.

Since

Version 1.0.0

Enumerator

k_session_validate_ok	Exact sequence match (most common case)
k_session_validate_gap	Accepted with gap (packet loss detected)
k_session_validate_fail	Rejected (large gap or duplicate)

Definition at line 153 of file rx_session.h.

```
00153             : uint8_t {
00154   k_session_validate_ok   = 0, /**< Exact sequence match (most common case) */
00155   k_session_validate_gap  = 1, /**< Accepted with gap (packet loss detected) */
00156   k_session_validate_fail = 2, /**< Rejected (large gap or duplicate) */
00157 } rx_session_validate_result_t;
```

4.1.4 Function Documentation

4.1.4.1 rx_session_deinit()

```
rx_err_t rx_session_deinit (
                rx_session_state_t * state)   [nodiscard]
```

Deinitialize session state and release resources.

Deletes the internal ThreadX mutex and marks the session as uninitialized. After this call, no other rx_session_*() function may be called until rx_session_init() is called again.

Parameters

in,out	state	Session state to deinitialize (must not be NULL)

Returns

rx_err_t Error code

Return values

k_rx_ok	Success, resources released
k_rx_err_null_ptr	state isnullptr
k_rx_err_not_initialized	state was not initialized

Precondition

state must be initialized via rx_session_init()

Postcondition

state->initialized == false

Internal mutex deleted

Note

Thread-safe: Caller must ensure no other threads are using the session

See also

rx_session_init() Initialization counterpart

Since

Version 1.0.0

Deletes the internal ThreadX mutex and marks the session as uninitialized. After this call, no other rx_session_*() function may be called until rx_session_init() is called again.

In simulator builds, mutex deletion is skipped since it was never created.

Precondition

state !=nullptr

state->initialized == true

Postcondition

state->initialized == false

Internal mutex deleted (on non-simulator builds)

Note

Thread-safe: Caller must ensure no other threads are using the session

See also

rx_session_init() Initialization counterpart

Since

Version 1.0.0

Definition at line 216 of file rx_session.c.

```
00217 {
00218   RX_CHECK_NULL_PTR(state, s_tag, "Deinit: state is NULL");
00219   if (!state->initialized) {
00220     rx_log_error(s_tag, "Deinit: session not initialized");
00221     return k_rx_err_not_initialized;
00222   }
00223
00224 #if !RX_IS_SIMULATOR
00225   /* tx_mutex_delete return value deliberately discarded: mutex deletion errors
00226    * are non-fatal during deinit because the mutex may already be uninitialized
00227    * or no recovery is possible. The module is being torn down regardless. */
00228   (void)tx_mutex_delete(&s_session_mutex);
00229 #endif
00230
00231   state->initialized = false;
00232
00233   rx_log_info(s_tag, "Session deinitialized");
00234   return k_rx_ok;
00235 }
```

References rx_session_state_t::initialized, s_session_mutex, and s_tag.

4.1.4.2 rx_session_get_rx()

```
rx_err_t rx_session_get_rx (
                const rx_session_state_t * state,
                uint16_t * sequence)   [nodiscard]
```

Get current expected RX sequence.

Returns the next expected RX sequence counter value for diagnostic purposes. Does not modify the counter.

Parameters

in	state	Session state (must be initialized)
out	sequence	Pointer to receive the current RX sequence

Returns

rx_err_t Error code

Return values

k_rx_ok	Success, sequence written
k_rx_err_null_ptr	state or sequence isnullptr
k_rx_err_not_initialized	state not initialized

Precondition

state must be initialized via rx_session_init()

Postcondition

state unchanged

Note

Thread-safe: Protected by internal mutex

See also

rx_session_validate_rx() Validate and update

Since

Version 1.0.0

Get current expected RX sequence.

Returns the next expected RX sequence counter value for diagnostic purposes. Does not modify the counter.

Precondition

state must be initialized via rx_session_init()

sequence must point to valid memory

Postcondition

state unchanged

*sequence <= k_session_seq_wrap_mask

Note

Thread-safe: Protected by internal mutex

See also

rx_session_validate_rx() Validate and update

Since

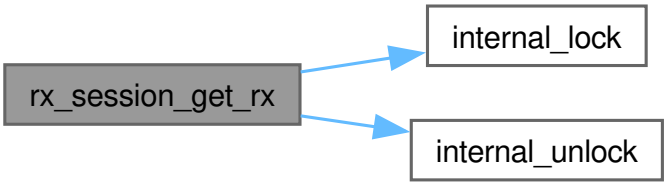
Version 1.0.0

Definition at line 464 of file rx_session.c.

```
00465 {
00466   RX_CHECK_NULL_PTR(state, s_tag, "GetRx: state is NULL");
00467   RX_CHECK_NULL_PTR(sequence, s_tag, "GetRx: sequence is NULL");
00468   if (!state->initialized) {
00469     rx_log_error(s_tag, "GetRx: session not initialized");
00470     return k_rx_err_not_initialized;
00471   }
00472
00473   rx_err_t lock_err = internal_lock();
00474   (void)lock_err;
00475
00476   *sequence = state->rx_sequence;
00477
00478   internal_unlock();
00479
00480   /* Post-condition: *sequence <= k_session_seq_wrap_mask.
00481    * Guaranteed by the invariant on rx_sequence; no runtime check needed. */
00482
00483   return k_rx_ok;
00484 }
```

References rx_session_state_t::initialized, internal_lock(), internal_unlock(), rx_session_state_t::rx_sequence, and s_tag.

Here is the call graph for this function:



4.1.4.3 rx_session_get_tx()

```
rx_err_t rx_session_get_tx (
                const rx_session_state_t * state,
                uint16_t * sequence)    [nodiscard]
```

Get current TX sequence without incrementing.

Returns the current TX sequence counter value for diagnostic purposes. Does not modify the counter.

Parameters

in	state	Session state (must be initialized)
out	sequence	Pointer to receive the current TX sequence

Returns

rx_err_t Error code

Return values

k_rx_ok	Success, sequence written
k_rx_err_null_ptr	state or sequence isnullptr
k_rx_err_not_initialized	state not initialized

Precondition

state must be initialized via rx_session_init()

Postcondition

state unchanged

Note

Thread-safe: Protected by internal mutex

See also

rx_session_next_tx() Read and increment

Since

Version 1.0.0

Returns the current TX sequence counter value for diagnostic purposes. Does not modify the counter.

Precondition

state must be initialized via rx_session_init()

sequence must point to valid memory

Postcondition

state unchanged

*sequence <= k_session_seq_wrap_mask

Note

Thread-safe: Protected by internal mutex

See also

rx_session_next_tx() Read and increment

Since

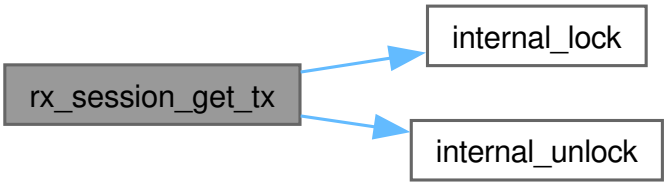
Version 1.0.0

Definition at line 423 of file rx_session.c.

```
00424 {
00425   RX_CHECK_NULL_PTR(state, s_tag, "GetTx: state is NULL");
00426   RX_CHECK_NULL_PTR(sequence, s_tag, "GetTx: sequence is NULL");
00427   if (!state->initialized) {
00428     rx_log_error(s_tag, "GetTx: session not initialized");
00429     return k_rx_err_not_initialized;
00430   }
00431
00432   rx_err_t lock_err = internal_lock();
00433   (void)lock_err;
00434
00435   *sequence = state->tx_sequence;
00436
00437   internal_unlock();
00438
00439   /* Post-condition: *sequence <= k_session_seq_wrap_mask.
00440    * Guaranteed by the invariant on tx_sequence; no runtime check needed. */
00441
00442   return k_rx_ok;
00443 }
```

References rx_session_state_t::initialized, internal_lock(), internal_unlock(), s_tag, and rx_session_state_t::tx_sequer

Here is the call graph for this function:



4.1.4.4 rx_session_init()

rx_err_t rx_session_init (
rx_session_state_t * state) [nodiscard]

Initialize session state with zeroed sequences.

Sets both TX and RX sequences to 0 and creates the internal ThreadX mutex for thread-safe access. Must be called before any other rx_session_*() function.

In simulator builds (RX_SIMULATOR_MODE), mutex creation is skipped since the ThreadX kernel is not running.

Parameters

in,out	state	Session state to initialize (must not be NULL)

Returns

rx_err_t Error code

Return values

k_rx_ok	Success, session ready for use
k_rx_err_null_ptr	state isnullptr
k_rx_err_rtos_mutex	ThreadX mutex creation failed

Precondition

state must point to valid, allocated memory

state must not already be initialized (call rx_session_deinit() first)

Postcondition

state->tx_sequence == 0

state->rx_sequence == 0

state->initialized == true

Note

Thread-safe: No mutex needed during init (single-threaded startup)

Example:

```
static rx_session_state_t g_session;
rx_err_t err = rx_session_init(&g_session);
assert(err == k_rx_ok);
```

See also

rx_session_deinit() Cleanup counterpart

Since

Version 1.0.0

Sets both TX and RX sequences to k_session_initial_sequence and creates the internal ThreadX mutex for thread-safe access. Must be called before any other rx_session_*() function.

In simulator builds (RX_SIMULATOR_MODE), mutex creation is skipped since the ThreadX kernel is not running.

Precondition

state must point to valid, allocated memory

state must not already be initialized (call rx_session_deinit() first)

Postcondition

state->tx_sequence == k_session_initial_sequence

state->rx_sequence == k_session_initial_sequence

state->initialized == true

Note

Thread-safe: No mutex needed during init (single-threaded startup)

See also

rx_session_deinit() Cleanup counterpart

Since

Version 1.0.0

Definition at line 171 of file rx_session.c.

```
00172 {
00173   RX_CHECK_NULL_PTR(state, s_tag, "Init: state is NULL");
00174
00175   state->tx_sequence = k_session_initial_sequence;
00176   state->rx_sequence = k_session_initial_sequence;
00177   state->initialized = true;
00178
00179 #if !RX_IS_SIMULATOR
00180   UINT status = tx_mutex_create(&s_session_mutex, "SessionMutex", TX_NO_INHERIT);
00181   if (status != TX_SUCCESS) {
00182     rx_log_error(s_tag, "Mutex creation failed");
00183     state->initialized = false;
00184     return k_rx_err_rtos_mutex;
00185   }
00186 #endif
00187
00188   /* Post-condition: verify initialization succeeded */
00189
00190   rx_log_info(s_tag, "Session initialized (tx=0, rx=0)");
00191   return k_rx_ok;
00192 }
```

References rx_session_state_t::initialized, k_session_initial_sequence, rx_session_state_t::rx_sequence, s_session_mutex, s_tag, and rx_session_state_t::tx_sequence.

4.1.4.5 rx_session_next_tx()

```
rx_err_t rx_session_next_tx (
                rx_session_state_t * state,
                uint16_t * sequence)   [nodiscard]
```

Get next TX sequence number and increment the counter.

Atomically reads the current TX sequence, increments it (with wraparound at 65535), and returns the pre-increment value. This is used by both USB and SPI transport layers when sending frames.

Mirrors Go gateway's SessionState.NextTxSequence().

Parameters

in,out	state	Session state (must be initialized)
out	sequence	Pointer to receive the sequence number

Returns

rx_err_t Error code

Return values

k_rx_ok	Success, sequence written
k_rx_err_null_ptr	state or sequence isnullptr
k_rx_err_not_initialized	state not initialized

Precondition

state must be initialized via rx_session_init()

sequence must point to valid memory

Postcondition

state->tx_sequence incremented by 1 (with wraparound)

*sequence contains the pre-increment value

Note

Thread-safe: Protected by internal mutex

Example:

```
uint16_t seq;
rx_err_t err = rx_session_next_tx(&g_session, &seq);
if (err == k_rx_ok) {
  frame.header.sequence = seq;
}
```

See also

rx_session_get_tx() Read without incrementing

Since

Version 1.0.0

Atomically reads the current TX sequence, increments it (with wraparound at k_session_seq_wrap_mask), and returns the pre-increment value. This is used by both USB and SPI transport layers when sending frames.

Mirrors Go gateway's SessionState.NextTxSequence().

Precondition

state must be initialized via rx_session_init()

sequence must point to valid memory

Postcondition

state->tx_sequence incremented by 1 (with wraparound)

*sequence contains the pre-increment value

*sequence <= k_session_seq_wrap_mask

Note

Thread-safe: Protected by internal mutex

See also

rx_session_get_tx() Read without incrementing

Since

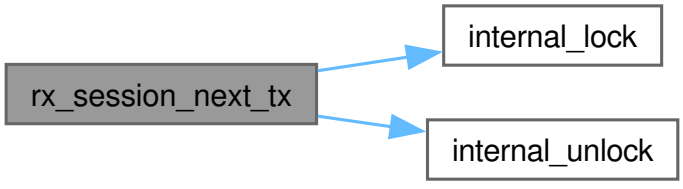
Version 1.0.0

Definition at line 260 of file rx_session.c.

```
00261 {
00262   RX_CHECK_NULL_PTR(state, s_tag, "NextTx: state is NULL");
00263   RX_CHECK_NULL_PTR(sequence, s_tag, "NextTx: sequence is NULL");
00264   if (!state->initialized) {
00265     rx_log_error(s_tag, "NextTx: session not initialized");
00266     return k_rx_err_not_initialized;
00267   }
00268
00269   rx_err_t lock_err = internal_lock();
00270   (void)lock_err;
00271
00272   *sequence          = state->tx_sequence;
00273   state->tx_sequence = (state->tx_sequence + 1) & k_session_seq_wrap_mask;
00274
00275   internal_unlock();
00276
00277   /* Post-condition: *sequence <= k_session_seq_wrap_mask.
00278    * Guaranteed by the bitwise AND above; no runtime check needed. */
00279
00280   return k_rx_ok;
00281 }
```

References rx_session_state_t::initialized, internal_lock(), internal_unlock(), k_session_seq_wrap_mask, s_tag, and rx_session_state_t::tx_sequence.

Here is the call graph for this function:



4.1.4.6 rx_session_reset()

```
rx_err_t rx_session_reset (
                rx_session_state_t * state)   [nodiscard]
```

Reset both TX and RX sequences to zero.

Called after a RESET/RESET_ACK handshake completes to synchronize sequences between firmware and gateway. Both counters are atomically set to 0.

Mirrors Go gateway's SessionState.Reset().

Parameters

in,out	state	Session state (must be initialized)

Returns

rx_err_t Error code

Return values

k_rx_ok	Success, sequences reset
k_rx_err_null_ptr	state isnullptr
k_rx_err_not_initialized	state not initialized

Precondition

state must be initialized via rx_session_init()

Postcondition

state->tx_sequence == 0

state->rx_sequence == 0

Note

Thread-safe: Protected by internal mutex

See also

rx_frame_type_t::k_frame_type_reset RESET frame triggers this

rx_frame_type_t::k_frame_type_reset_ack RESET_ACK confirms reset

Since

Version 1.0.0

Reset both TX and RX sequences to zero.

Called after a RESET/RESET_ACK handshake completes to synchronize sequences between firmware and gateway. Both counters are atomically set to k_session_initial_sequence.

Mirrors Go gateway's SessionState.Reset().

Precondition

state must be initialized via rx_session_init()

Postcondition

state->tx_sequence == k_session_initial_sequence

state->rx_sequence == k_session_initial_sequence

Note

Thread-safe: Protected by internal mutex

See also

rx_frame_type_t::k_frame_type_reset RESET frame triggers this

rx_frame_type_t::k_frame_type_reset_ack RESET_ACK confirms reset

Since

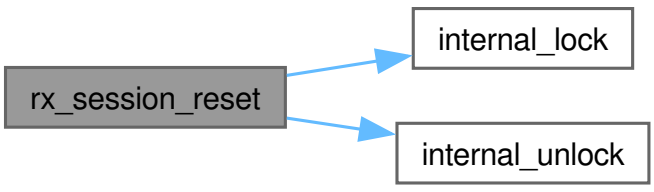
Version 1.0.0

Definition at line 384 of file rx_session.c.

```
00385 {
00386   RX_CHECK_NULL_PTR(state, s_tag, "Reset: state is NULL");
00387   if (!state->initialized) {
00388     rx_log_error(s_tag, "Reset: session not initialized");
00389     return k_rx_err_not_initialized;
00390   }
00391
00392   rx_err_t lock_err = internal_lock();
00393   (void)lock_err;
00394
00395   state->tx_sequence = k_session_initial_sequence;
00396   state->rx_sequence = k_session_initial_sequence;
00397
00398   internal_unlock();
00399
00400   rx_log_info(s_tag, "Session reset (tx=0, rx=0)");
00401   return k_rx_ok;
00402 }
```

References rx_session_state_t::initialized, internal_lock(), internal_unlock(), k_session_initial_sequence, rx_session_state_t::rx_sequence, s_tag, and rx_session_state_t::tx_sequence.

Here is the call graph for this function:



4.1.4.7 rx_session_validate_rx()

```
rx_err_t rx_session_validate_rx (
                rx_session_state_t * state,
                uint16_t received_seq,
                rx_session_validate_result_t * result)   [nodiscard]
```

Validate a received sequence number.

Checks whether the received sequence number is acceptable based on the expected RX sequence. Implements gap tolerance matching the Go gateway's SessionState.ValidateRxSequence().

4.1.4.8 Validation Logic

Condition	Result	RX Sequence Update
diff == 0	k_session_validate_ok	rx_sequence = received + 1
0 < diff < k_session_max_gap_tolerance	k_session_validate_gap	rx_sequence = received + 1
diff >= k_session_max_gap_tolerance	k_session_validate_fail	rx_sequence unchanged
diff wraps near 65535 (duplicate)	k_session_validate_fail	rx_sequence unchanged

The diff is computed as received_seq - rx_sequence using unsigned 16-bit arithmetic, which handles wraparound correctly.

Parameters

in,out	state	Session state (must be initialized)
in	received_seq	The sequence number from the received frame
out	result	Validation result code (may be NULL if not needed)

Returns

rx_err_t Error code

Return values

k_rx_ok	Sequence accepted (exact match or small gap)
k_rx_err_protocol_error	Sequence rejected (large gap or duplicate)
k_rx_err_null_ptr	state isnullptr
k_rx_err_not_initialized	state not initialized

Precondition

state must be initialized via rx_session_init()

Postcondition

On accept: state->rx_sequence updated to received_seq + 1

On reject: state->rx_sequence unchanged

Note

Thread-safe: Protected by internal mutex

Example:

```
rx_session_validate_result_t result;
rx_err_t err = rx_session_validate_rx(&g_session, frame.header.sequence, &result);
if (err == k_rx_ok) {
  if (result == k_session_validate_gap) {
    rx_log_warn("SESSION", "Gap detected in sequence");
  }
  process_frame(&frame);
}
```

See also

k_session_max_gap_tolerance Maximum acceptable gap

Since

Version 1.0.0

Validate a received sequence number.

Checks whether the received sequence number is acceptable based on the expected RX sequence. Implements gap tolerance matching the Go gateway's SessionState.ValidateRxSequence().

4.1.4.9 Validation Logic

Condition	Result	RX Sequence Update
diff == k_session_diff_exact_match	k_session_validate_ok	rx_sequence = received + 1
0 < diff < k_session_max_gap_tolerance	k_session_validate_gap	rx_sequence = received + 1
diff >= k_session_max_gap_tolerance	k_session_validate_fail	rx_sequence unchanged

Precondition

state must be initialized via rx_session_init()

Postcondition

On accept: state->rx_sequence updated to received_seq + 1

On reject: state->rx_sequence unchanged

Note

Thread-safe: Protected by internal mutex

See also

k_session_max_gap_tolerance Maximum acceptable gap

Since

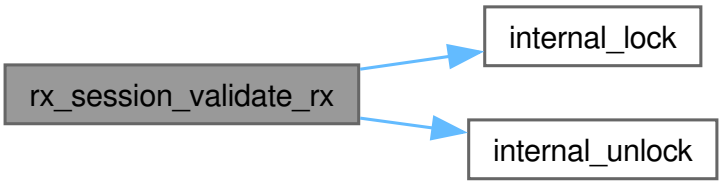
Version 1.0.0

Definition at line 310 of file rx_session.c.

```
00313 {
00314   RX_CHECK_NULL_PTR(state, s_tag, "ValidateRx: state is NULL");
00315   if (!state->initialized) {
00316     rx_log_error(s_tag, "ValidateRx: session not initialized");
00317     return k_rx_err_not_initialized;
00318   }
00319
00320   rx_err_t lock_err = internal_lock();
00321   (void)lock_err;
00322
00323   /* Calculate difference using unsigned 16-bit arithmetic (handles wraparound) */
00324   uint16_t diff = received_seq - state->rx_sequence;
00325
00326   /* Exact match - most common case */
00327   if (diff == k_session_diff_exact_match) {
00328     state->rx_sequence = (state->rx_sequence + 1) & k_session_seq_wrap_mask;
00329     internal_unlock();
00330
00331     if (result != NULL) {
00332       *result = k_session_validate_ok;
00333     }
00334     return k_rx_ok;
00335   }
00336
00337   /* Small gap (packet loss) - accept and catch up.
00338    * diff > k_session_diff_exact_match is always true here: the exact-match case
00339    * already returned above, so the condition reduces to a single comparison. */
00340   if (diff < k_session_max_gap_tolerance) {
00341     rx_log_warn_val(s_tag, "Sequence gap detected, frames lost", diff);
00342     state->rx_sequence = (received_seq + 1) & k_session_seq_wrap_mask;
00343     internal_unlock();
00344
00345     if (result != NULL) {
00346       *result = k_session_validate_gap;
00347     }
00348     return k_rx_ok;
00349   }
00350
00351   /* Large gap or duplicate (diff >= gap tolerance or near-65535 wraparound) */
00352   rx_log_error_val(s_tag, "Sequence rejected", received_seq);
00353
00354   internal_unlock();
00355
00356   if (result != NULL) {
00357     *result = k_session_validate_fail;
00358   }
00359   return k_rx_err_protocol_error;
00360 }
```

References rx_session_state_t::initialized, internal_lock(), internal_unlock(), k_session_diff_exact_match, k_session_max_gap_tolerance, k_session_seq_wrap_mask, k_session_validate_fail, k_session_validate_gap, k_session_validate_ok, rx_session_state_t::rx_sequence, and s_tag.

Here is the call graph for this function:



4.2 rx_session.h

Go to the documentation of this file.

```
00001 /**
00002  * @file rx_session.h
00003  *
00004  * @brief Shared session state for cross-transport sequence continuity
00005  *
00006  * @details
00007  * Provides a single, shared session state (TX and RX sequence counters) that is
00008  * used by BOTH USB CDC and SPI transport layers. When the active transport
00009  * switches (e.g., USB at seq=105 to SPI), SPI continues at seq=106 because
00010  * both transports reference the same session state.
00011  *
00012  * This module mirrors the Go gateway's `manager/session.go` implementation:
00013  * - `NextTxSequence()` -> `rx_session_next_tx()`
00014  * - `ValidateRxSequence()` -> `rx_session_validate_rx()`
00015  * - `Reset()` -> `rx_session_reset()`
00016  *
00017  * ## Architecture
00018  *
00019  * ```
00020  * rx_comm_manager_t (owns rx_session_state_t)
00021  *       |
00022  *       +-- rx_usb_comm_handle_t.session --> &session_state
00023  *       |
00024  *       +-- rx_spi_comm_handle_t.session --> &session_state
00025  * ```
00026  *
00027  * ## Thread Safety
00028  *
00029  * All public functions are thread-safe. Internal state is protected by a
00030  * ThreadX mutex, allowing safe concurrent access from the comm_task thread,
00031  * USB ISR callbacks, and SPI ISR callbacks.
00032  *
00033  * ## Gap Tolerance (Matching Go Gateway)
00034  *
00035  * The RX sequence validator accepts small gaps (up to 10 frames) to handle
00036  * occasional packet loss on the lightweight USB CDC transport. This matches
00037  * the Go gateway's `MaxGapTolerance = 10` behavior:
00038  *
00039  * | Condition | Action |
00040  * |---|---|
00041  * | diff == 0 | Accept, increment rx_sequence |
00042  * | 0 < diff < 10 | Accept with warning, update rx_sequence |
00043  * | diff >= 10 | Reject, log error |
00044  *
00045  * @par NASA Power of 10 Compliance:
00046  * - Rule 1: No goto/setjmp/longjmp/recursion
00047  * - Rule 2: Fixed loop bounds (no loops in this module)
00048  * - Rule 3: No dynamic allocation (caller provides state struct)
00049  * - Rule 5: Precondition/postcondition validation on all functions
00050  * - Rule 7: All return values checked
00051  * - Rule 8: Macros only for conditional compilation
00052  * - Rule 10: Compiled with -Wall -Wextra -Werror
00053  *
00054  * @see star-gateway/internal/manager/session.go  Go reference implementation
00055  * @see rx_usb_comm.h  USB CDC transport (uses session for sequence tracking)
00056  * @see rx_spi_comm.h  SPI transport (uses session for sequence tracking)
00057  * @see rx_comm_manager.h  Communication manager (owns session instance)
00058  *
00059  * @since Version 1.0.0
00060  *
00061  * @par Locked, Inc.
00062  * @par February 2026
00063
00064  * @copyright Copyright (c) 2026 Locked Inc.
00065  * SPDX-License-Identifier: MIT
00066 */
00067
00068 #pragma once
00069
00070 #include <stdbool.h>
00071
00072 /*
      ================================================================================
00073  * Includes
00074  *
      ================================================================================
      */
00075
00076 #include <stdint.h>
00077
00078 #include "rx_err.h"
00079
```

```
00080 /*
      ============================================================================
00081  * Constants
00082  *
      ============================================================================
      */
00083
00084 /**
00085  * @enum rx_session_constants_t
00086  * @brief Session state configuration constants
00087  *
00088  * @details
00089  * Configuration parameters for the session state module. These match the
00090  * Go gateway's constants in `manager/session.go`.
00091  *
00092  * @see star-gateway/internal/manager/session.go  Go reference values
00093  *
00094  * @since Version 1.0.0
00095  */
00096 typedef enum : uint16_t {
00097     /**
00098      * @brief Initial TX/RX sequence value after init or reset
00099      *
00100      * @details
00101      * Both TX and RX sequence counters start at this value. Used by
00102      * rx_session_init() and rx_session_reset().
00103      */
00104     k_session_initial_sequence = 0,
00105
00106     /**
00107      * @brief Exact-match diff value for sequence validation
00108      *
00109      * @details
00110      * When the difference between received and expected sequence is zero,
00111      * the sequence matches exactly (most common case).
00112      */
00113     k_session_diff_exact_match = 0,
00114
00115     /**
00116      * @brief Maximum allowable gap between expected and received sequence numbers
00117      *
00118      * @details
00119      * When the received sequence is ahead of expected by this amount or less,
00120      * the frame is accepted and the RX sequence catches up. Gaps larger than
00121      * this are rejected as transport failures.
00122      *
00123      * Set to half the 16-bit wrap window so any "ahead" sequence is accepted
00124      * (only true past-window duplicates are rejected). This relaxed bound
00125      * lets ad-hoc bring-up tools (cmd_debug, vel_test, grpcurl one-shots)
00126      * drive the link without re-implementing the full session library.
00127      * The production gateway tracks sequences precisely via rx_session and
00128      * stays well under this limit, so the relaxation is invisible to it.
00129      */
00130     k_session_max_gap_tolerance = 32768,
00131
00132     /**
00133      * @brief Sequence number wraparound mask (16-bit unsigned)
00134      *
00135      * @details
00136      * Applied after incrementing sequence counters to ensure they wrap
00137      * around at 65535 back to 0. Matches Go: `(seq + 1) & 0xFFFF`
00138      */
00139     k_session_seq_wrap_mask = 0xFFFF,
00140 } rx_session_constants_t;
00141
00142 /**
00143  * @enum rx_session_validate_result_t
00144  * @brief Result codes for sequence validation
00145  *
00146  * @details
00147  * Returned by `rx_session_validate_rx()` to indicate the validation outcome.
00148  * This provides more information than a simple bool, allowing callers to
00149  * distinguish between exact matches, gap acceptance, and rejection.
00150  *
00151  * @since Version 1.0.0
00152  */
00153 typedef enum : uint8_t {
00154     k_session_validate_ok   = 0, /**< Exact sequence match (most common case) */
00155     k_session_validate_gap  = 1, /**< Accepted with gap (packet loss detected) */
00156     k_session_validate_fail = 2, /**< Rejected (large gap or duplicate) */
00157 } rx_session_validate_result_t;
00158
00159 /*
      ============================================================================
00160  * Types
00161  *
      ============================================================================
```

```
     */
00162
00163 /**
00164  * @struct rx_session_state_t
00165  * @brief Shared session state for cross-transport sequence continuity
00166  *
00167  * @details
00168  * Holds TX and RX sequence counters shared between USB CDC and SPI transports.
00169  * The `rx_comm_manager_t` owns a single instance and passes a pointer to both
00170  * transport handles during initialization.
00171  *
00172  * ## Memory Layout (8 bytes, no padding)
00173  *
00174  * | Offset | Size | Field | Description |
00175  * |--------|------|-------|-------------|
00176  * | 0x00 | 2 B | tx_sequence | Next TX sequence to assign |
00177  * | 0x02 | 2 B | rx_sequence | Next expected RX sequence |
00178  * | 0x04 | 1 B | initialized | Initialization flag |
00179  * | 0x05 | 3 B | (padding) | Alignment padding |
00180  *
00181  * @invariant tx_sequence and rx_sequence are only modified under mutex protection
00182  * @invariant initialized is true after successful rx_session_init()
00183  *
00184  * @par Example:
00185  * @code{.c}
00186  * // Owned by comm manager, shared with both transports
00187  * static rx_session_state_t g_session;
00188  *
00189  * rx_err_t err = rx_session_init(&g_session);
00190  * if (err != k_rx_ok) {
00191  *   rx_log_error("SESSION", "Init failed");
00192  *   return err;
00193  * }
00194  *
00195  * // Pass to both transport handles
00196  * usb_handle.session = &g_session;
00197  * spi_handle.session = &g_session;
00198  * @endcode
00199  *
00200  * @note Do not access fields directly. Use the rx_session_*() API functions.
00201  * @warning This struct must be statically allocated (no malloc).
00202  *
00203  * @see rx_session_init() Initialize before use
00204  * @see rx_session_deinit() Cleanup when done
00205  *
00206  * @since Version 1.0.0
00207  */
00208 typedef struct {
00209   uint16_t tx_sequence; /**< Next TX sequence number to assign (wraps at 65535) */
00210   uint16_t rx_sequence; /**< Next expected RX sequence number (wraps at 65535) */
00211   bool     initialized; /**< True after successful rx_session_init() */
00212 } rx_session_state_t;
00213
00214 /*
     ================================================================================
00215  * Public API
00216  *
     ================================================================================
     */
00217
00218 /**
00219  * @brief Initialize session state with zeroed sequences
00220  *
00221  * @details
00222  * Sets both TX and RX sequences to 0 and creates the internal ThreadX mutex
00223  * for thread-safe access. Must be called before any other rx_session_*()
00224  * function.
00225  *
00226  * In simulator builds (RX_SIMULATOR_MODE), mutex creation is skipped since
00227  * the ThreadX kernel is not running.
00228  *
00229  * @param[in,out] state Session state to initialize (must not be NULL)
00230  *
00231  * @return rx_err_t Error code
00232  * @retval k_rx_ok Success, session ready for use
00233  * @retval k_rx_err_null_ptr state isnullptr
00234  * @retval k_rx_err_rtos_mutex ThreadX mutex creation failed
00235  *
00236  * @pre state must point to valid, allocated memory
00237  * @pre state must not already be initialized (call rx_session_deinit() first)
00238  * @post state->tx_sequence == 0
00239  * @post state->rx_sequence == 0
00240  * @post state->initialized == true
00241  *
00242  * @note Thread-safe: No mutex needed during init (single-threaded startup)
00243  *
00244  * @par Example:
```

```
00245  * @code{.c}
00246  * static rx_session_state_t g_session;
00247  * rx_err_t err = rx_session_init(&g_session);
00248  * assert(err == k_rx_ok);
00249  * @endcode
00250  *
00251  * @see rx_session_deinit() Cleanup counterpart
00252  *
00253  * @since Version 1.0.0
00254  */
00255 [[nodiscard]] rx_err_t rx_session_init(rx_session_state_t* state);
00256
00257 /**
00258  * @brief Deinitialize session state and release resources
00259  *
00260  * @details
00261  * Deletes the internal ThreadX mutex and marks the session as uninitialized.
00262  * After this call, no other rx_session_*() function may be called until
00263  * rx_session_init() is called again.
00264  *
00265  * @param[in,out] state Session state to deinitialize (must not be NULL)
00266  *
00267  * @return rx_err_t Error code
00268  * @retval k_rx_ok Success, resources released
00269  * @retval k_rx_err_null_ptr state isnullptr
00270  * @retval k_rx_err_not_initialized state was not initialized
00271  *
00272  * @pre state must be initialized via rx_session_init()
00273  * @post state->initialized == false
00274  * @post Internal mutex deleted
00275  *
00276  * @note Thread-safe: Caller must ensure no other threads are using the session
00277  *
00278  * @see rx_session_init() Initialization counterpart
00279  *
00280  * @since Version 1.0.0
00281  */
00282 [[nodiscard]] rx_err_t rx_session_deinit(rx_session_state_t* state);
00283
00284 /**
00285  * @brief Get next TX sequence number and increment the counter
00286  *
00287  * @details
00288  * Atomically reads the current TX sequence, increments it (with wraparound
00289  * at 65535), and returns the pre-increment value. This is used by both USB
00290  * and SPI transport layers when sending frames.
00291  *
00292  * Mirrors Go gateway's `SessionState.NextTxSequence()`.
00293  *
00294  * @param[in,out] state Session state (must be initialized)
00295  * @param[out] sequence Pointer to receive the sequence number
00296  *
00297  * @return rx_err_t Error code
00298  * @retval k_rx_ok Success, sequence written
00299  * @retval k_rx_err_null_ptr state or sequence isnullptr
00300  * @retval k_rx_err_not_initialized state not initialized
00301  *
00302  * @pre state must be initialized via rx_session_init()
00303  * @pre sequence must point to valid memory
00304  * @post state->tx_sequence incremented by 1 (with wraparound)
00305  * @post *sequence contains the pre-increment value
00306  *
00307  * @note Thread-safe: Protected by internal mutex
00308  *
00309  * @par Example:
00310  * @code{.c}
00311  * uint16_t seq;
00312  * rx_err_t err = rx_session_next_tx(&g_session, &seq);
00313  * if (err == k_rx_ok) {
00314  *   frame.header.sequence = seq;
00315  * }
00316  * @endcode
00317  *
00318  * @see rx_session_get_tx() Read without incrementing
00319  *
00320  * @since Version 1.0.0
00321  */
00322 [[nodiscard]] rx_err_t rx_session_next_tx(rx_session_state_t* state, uint16_t* sequence);
00323
00324 /**
00325  * @brief Validate a received sequence number
00326  *
00327  * @details
00328  * Checks whether the received sequence number is acceptable based on the
00329  * expected RX sequence. Implements gap tolerance matching the Go gateway's
00330  * `SessionState.ValidateRxSequence()`.
00331  *
```

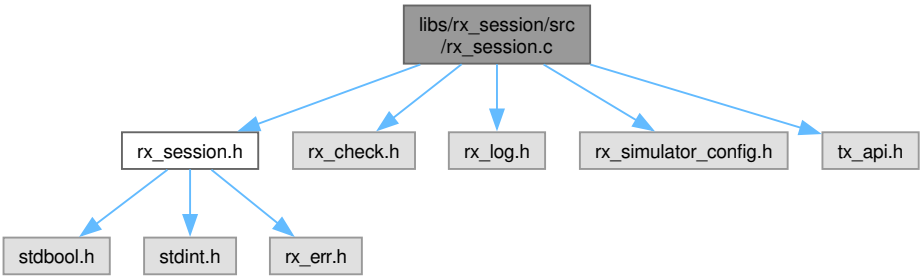
```
00332  * ## Validation Logic
00333  *
00334  * | Condition | Result | RX Sequence Update |
00335  * |---|---|---|
00336  * | diff == 0 | k_session_validate_ok | rx_sequence = received + 1 |
00337  * | 0 < diff < k_session_max_gap_tolerance | k_session_validate_gap | rx_sequence = received + 1 |
00338  * | diff >= k_session_max_gap_tolerance | k_session_validate_fail | rx_sequence unchanged |
00339  * | diff wraps near 65535 (duplicate) | k_session_validate_fail | rx_sequence unchanged |
00340  *
00341  * The diff is computed as `received_seq - rx_sequence` using unsigned 16-bit
00342  * arithmetic, which handles wraparound correctly.
00343  *
00344  * @param[in,out] state Session state (must be initialized)
00345  * @param[in] received_seq The sequence number from the received frame
00346  * @param[out] result Validation result code (may be NULL if not needed)
00347  *
00348  * @return rx_err_t Error code
00349  * @retval k_rx_ok Sequence accepted (exact match or small gap)
00350  * @retval k_rx_err_protocol_error Sequence rejected (large gap or duplicate)
00351  * @retval k_rx_err_null_ptr state isnullptr
00352  * @retval k_rx_err_not_initialized state not initialized
00353  *
00354  * @pre state must be initialized via rx_session_init()
00355  * @post On accept: state->rx_sequence updated to received_seq + 1
00356  * @post On reject: state->rx_sequence unchanged
00357  *
00358  * @note Thread-safe: Protected by internal mutex
00359  *
00360  * @par Example:
00361  * @code{.c}
00362  * rx_session_validate_result_t result;
00363  * rx_err_t err = rx_session_validate_rx(&g_session, frame.header.sequence, &result);
00364  * if (err == k_rx_ok) {
00365  *   if (result == k_session_validate_gap) {
00366  *     rx_log_warn("SESSION", "Gap detected in sequence");
00367  *   }
00368  *   process_frame(&frame);
00369  * }
00370  * @endcode
00371  *
00372  * @see k_session_max_gap_tolerance Maximum acceptable gap
00373  *
00374  * @since Version 1.0.0
00375  */
00376 [[nodiscard]] rx_err_t rx_session_validate_rx(rx_session_state_t*          state,
00377                                               uint16_t                    received_seq,
00378                                               rx_session_validate_result_t* result);
00379
00380 /**
00381  * @brief Reset both TX and RX sequences to zero
00382  *
00383  * @details
00384  * Called after a RESET/RESET_ACK handshake completes to synchronize sequences
00385  * between firmware and gateway. Both counters are atomically set to 0.
00386  *
00387  * Mirrors Go gateway's `SessionState.Reset()`.
00388  *
00389  * @param[in,out] state Session state (must be initialized)
00390  *
00391  * @return rx_err_t Error code
00392  * @retval k_rx_ok Success, sequences reset
00393  * @retval k_rx_err_null_ptr state isnullptr
00394  * @retval k_rx_err_not_initialized state not initialized
00395  *
00396  * @pre state must be initialized via rx_session_init()
00397  * @post state->tx_sequence == 0
00398  * @post state->rx_sequence == 0
00399  *
00400  * @note Thread-safe: Protected by internal mutex
00401  *
00402  * @see rx_frame_type_t::k_frame_type_reset RESET frame triggers this
00403  * @see rx_frame_type_t::k_frame_type_reset_ack RESET_ACK confirms reset
00404  *
00405  * @since Version 1.0.0
00406  */
00407 [[nodiscard]] rx_err_t rx_session_reset(rx_session_state_t* state);
00408
00409 /**
00410  * @brief Get current TX sequence without incrementing
00411  *
00412  * @details
00413  * Returns the current TX sequence counter value for diagnostic purposes.
00414  * Does not modify the counter.
00415  *
00416  * @param[in] state Session state (must be initialized)
00417  * @param[out] sequence Pointer to receive the current TX sequence
00418  *
```

```
00419  * @return rx_err_t Error code
00420  * @retval k_rx_ok Success, sequence written
00421  * @retval k_rx_err_null_ptr state or sequence isnullptr
00422  * @retval k_rx_err_not_initialized state not initialized
00423  *
00424  * @pre state must be initialized via rx_session_init()
00425  * @post state unchanged
00426  *
00427  * @note Thread-safe: Protected by internal mutex
00428  *
00429  * @see rx_session_next_tx() Read and increment
00430  *
00431  * @since Version 1.0.0
00432  */
00433 [[nodiscard]] rx_err_t rx_session_get_tx(const rx_session_state_t* state, uint16_t* sequence);
00434
00435 /**
00436  * @brief Get current expected RX sequence
00437  *
00438  * @details
00439  * Returns the next expected RX sequence counter value for diagnostic purposes.
00440  * Does not modify the counter.
00441  *
00442  * @param[in] state Session state (must be initialized)
00443  * @param[out] sequence Pointer to receive the current RX sequence
00444  *
00445  * @return rx_err_t Error code
00446  * @retval k_rx_ok Success, sequence written
00447  * @retval k_rx_err_null_ptr state or sequence isnullptr
00448  * @retval k_rx_err_not_initialized state not initialized
00449  *
00450  * @pre state must be initialized via rx_session_init()
00451  * @post state unchanged
00452  *
00453  * @note Thread-safe: Protected by internal mutex
00454  *
00455  * @see rx_session_validate_rx() Validate and update
00456  *
00457  * @since Version 1.0.0
00458  */
00459 [[nodiscard]] rx_err_t rx_session_get_rx(const rx_session_state_t* state, uint16_t* sequence);
00460
00461 /* end of rx_session.h */
```

4.3 libs/rx_session/src/rx_session.c File Reference

Shared session state implementation for cross-transport sequence continuity.

#include ”rx_session.h”
#include ”rx_check.h”
#include ”rx_log.h”
#include ”rx_simulator_config.h”
#include ”tx_api.h”
Include dependency graph for rx_session.c:



Functions

- static rx_err_t internal_lock (void)

Acquire the session mutex.

- static void internal_unlock (void)

  Release the session mutex.
- rx_err_t rx_session_init (rx_session_state_t *state)

  Initialize session state with zeroed sequences.
- rx_err_t rx_session_deinit (rx_session_state_t *state)

  Deinitialize session state and release resources.
- rx_err_t rx_session_next_tx (rx_session_state_t *state, uint16_t *sequence)

  Get next TX sequence number and increment the counter.
- rx_err_t rx_session_validate_rx (rx_session_state_t *state, uint16_t received_seq, rx_session_validate_result_ *result)

  Validate a received sequence number against expected RX sequence.
- rx_err_t rx_session_reset (rx_session_state_t *state)

  Reset both TX and RX sequences to initial values.
- rx_err_t rx_session_get_tx (const rx_session_state_t *state, uint16_t *sequence)

  Get current TX sequence without incrementing.
- rx_err_t rx_session_get_rx (const rx_session_state_t *state, uint16_t *sequence)

  Get current expected RX sequence without modifying.

Variables

- static const char s_tag [] = "SESSION"

  Logging tag for session state messages.
- static TX_MUTEX s_session_mutex

  ThreadX mutex for protecting session state access.

4.3.1 Detailed Description

Shared session state implementation for cross-transport sequence continuity.

Implements the cross-transport shared session state module. Provides thread-safe TX/RX sequence tracking with gap tolerance, mirroring the Go gateway's manager/session.go implementation.

Both USB CDC and SPI transports share a single session state instance (owned by rx_comm_manager_t), ensuring sequence continuity when the active transport switches.

4.3.1.1 Thread Safety

All public functions acquire the internal ThreadX mutex before accessing state. The mutex uses TX_NO_INHERIT (no priority inheritance) since contention is expected to be low (comm_task is the primary consumer).

In simulator builds (RX_SIMULATOR_MODE), mutex operations are skipped since the ThreadX kernel is not running.

Precondition

Caller must call rx_session_init() before any other API function.

Postcondition

After rx_session_deinit(), no API calls may be made until re-init.

NASA Power of 10 Compliance:

- Rule 1: No goto/setjmp/longjmp/recursion
- Rule 5: RX_CHECK_NULL_PTR / RX_VALIDATE_INIT preconditions on all functions
- Rule 7: All return values checked (RX_RETURN_ON_ERROR)
- Rule 8: Macros only for conditional compilation and validation

SOLID Compliance:

- S: Single responsibility (sequence tracking only)
- D: Depends on rx_err_t abstraction, not concrete transport details

Author

Locked, Inc. Team

Date

February 2026

Version

1.0.0

Copyright

See also

rx_session.h Public API

star-gateway/internal/manager/session.go Go reference implementation

Since

Version 1.0.0

Definition in file rx_session.c.

4.3.2 Function Documentation

4.3.2.1 internal_lock()

```
rx_err_t internal_lock (
            void )    [static]
```

Acquire the session mutex.

Locks the internal mutex with TX_WAIT_FOREVER. In simulator mode, this is a no-op since ThreadX is not running.

Precondition

Mutex must have been created via rx_session_init()

Postcondition

Mutex is held by calling thread

Note

Thread-safe: This IS the synchronization primitive

Since

Version 1.0.0

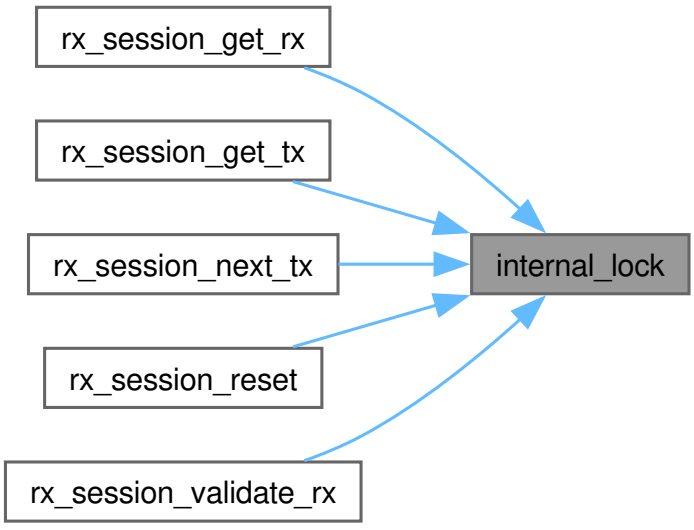
Definition at line 112 of file rx_session.c.

```
00113 {
00114 #if !RX_IS_SIMULATOR
00115   UINT status = tx_mutex_get(&s_session_mutex, TX_WAIT_FOREVER);
00116   if (status != TX_SUCCESS) {
00117     rx_log_error(s_tag, "Mutex lock failed");
00118     return k_rx_err_rtos_mutex;
00119   }
00120 #endif
00121   return k_rx_ok;
00122 }
```

References s_session_mutex, and s_tag.

Referenced by rx_session_get_rx(), rx_session_get_tx(), rx_session_next_tx(), rx_session_reset(), and rx_session_validate_rx().

Here is the caller graph for this function:



4.3.2.2 internal_unlock()

```
void internal_unlock (
                void ) [static]
```

Release the session mutex.

Unlocks the internal mutex. In simulator mode, this is a no-op.

Precondition

Mutex must be held by calling thread

Postcondition

Mutex is released

Note

Thread-safe: This IS the synchronization primitive

Since

Version 1.0.0

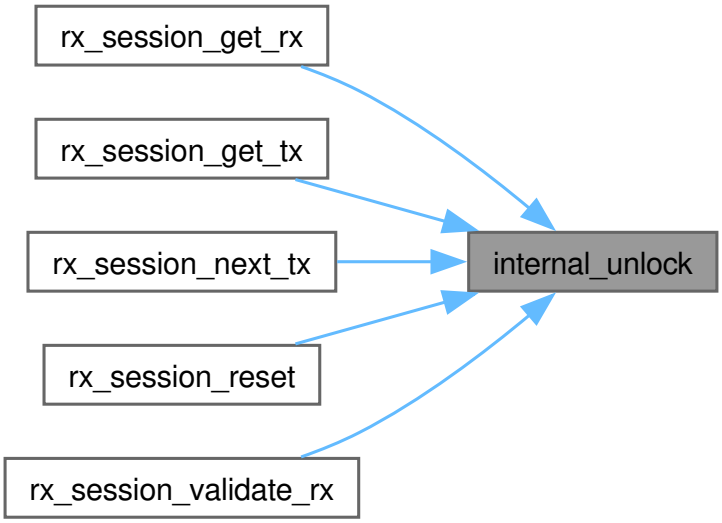
Definition at line 136 of file rx_session.c.

```
00137 {
00138 #if !RX_IS_SIMULATOR
00139   (void)tx_mutex_put(&s_session_mutex);
00140 #endif
00141 }
```

References s_session_mutex.

Referenced by rx_session_get_rx(), rx_session_get_tx(), rx_session_next_tx(), rx_session_reset(), and rx_session_validate_rx().

Here is the caller graph for this function:



4.3.2.3 rx_session_deinit()

```
rx_err_t rx_session_deinit (
                rx_session_state_t * state)   [nodiscard]
```

Deinitialize session state and release resources.

Deletes the internal ThreadX mutex and marks the session as uninitialized. After this call, no other rx_session_*() function may be called until rx_session_init() is called again.

In simulator builds, mutex deletion is skipped since it was never created.

Precondition

state !=nullptr

state->initialized == true

Postcondition

state->initialized == false

Internal mutex deleted (on non-simulator builds)

Note

Thread-safe: Caller must ensure no other threads are using the session

See also

rx_session_init() Initialization counterpart

Since

Version 1.0.0

Definition at line 216 of file rx_session.c.

```
00217 {
00218   RX_CHECK_NULL_PTR(state, s_tag, "Deinit: state is NULL");
00219   if (!state->initialized) {
00220     rx_log_error(s_tag, "Deinit: session not initialized");
00221     return k_rx_err_not_initialized;
00222   }
00223
00224 #if !RX_IS_SIMULATOR
00225   /* tx_mutex_delete return value deliberately discarded: mutex deletion errors
00226    * are non-fatal during deinit because the mutex may already be uninitialized
00227    * or no recovery is possible. The module is being torn down regardless. */
00228   (void)tx_mutex_delete(&s_session_mutex);
00229 #endif
00230
00231   state->initialized = false;
00232
00233   rx_log_info(s_tag, "Session deinitialized");
00234   return k_rx_ok;
00235 }
```

References rx_session_state_t::initialized, s_session_mutex, and s_tag.

4.3.2.4 rx_session_get_rx()

```
rx_err_t rx_session_get_rx (
                const rx_session_state_t * state,
                uint16_t * sequence)    [nodiscard]
```

Get current expected RX sequence without modifying.

Get current expected RX sequence.

Returns the next expected RX sequence counter value for diagnostic purposes. Does not modify the counter.

Precondition

state must be initialized via rx_session_init()

sequence must point to valid memory

Postcondition

state unchanged

*sequence <= k_session_seq_wrap_mask

Note

Thread-safe: Protected by internal mutex

See also

rx_session_validate_rx() Validate and update

Since

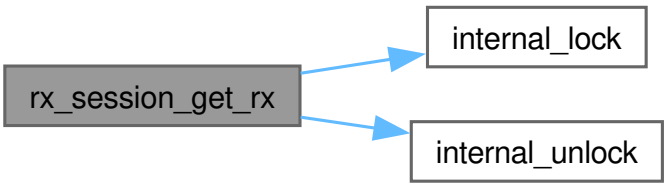
Version 1.0.0

Definition at line 464 of file rx_session.c.

```
00465 {
00466   RX_CHECK_NULL_PTR(state, s_tag, "GetRx: state is NULL");
00467   RX_CHECK_NULL_PTR(sequence, s_tag, "GetRx: sequence is NULL");
00468   if (!state->initialized) {
00469     rx_log_error(s_tag, "GetRx: session not initialized");
00470     return k_rx_err_not_initialized;
00471   }
00472
00473   rx_err_t lock_err = internal_lock();
00474   (void)lock_err;
00475
00476   *sequence = state->rx_sequence;
00477
00478   internal_unlock();
00479
00480   /* Post-condition: *sequence <= k_session_seq_wrap_mask.
00481    * Guaranteed by the invariant on rx_sequence; no runtime check needed. */
00482
00483   return k_rx_ok;
00484 }
```

References rx_session_state_t::initialized, internal_lock(), internal_unlock(), rx_session_state_t::rx_sequence, and s_tag.

Here is the call graph for this function:



4.3.2.5 rx_session_get_tx()

```
rx_err_t rx_session_get_tx (
                const rx_session_state_t * state,
                uint16_t * sequence)    [nodiscard]
```

Get current TX sequence without incrementing.

Returns the current TX sequence counter value for diagnostic purposes. Does not modify the counter.

Precondition

state must be initialized via rx_session_init()

sequence must point to valid memory

Postcondition

state unchanged

*sequence <= k_session_seq_wrap_mask

Note

Thread-safe: Protected by internal mutex

See also

rx_session_next_tx() Read and increment

Since

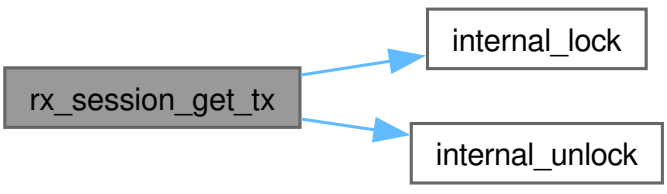
Version 1.0.0

Definition at line 423 of file rx_session.c.

```
00424 {
00425   RX_CHECK_NULL_PTR(state, s_tag, "GetTx: state is NULL");
00426   RX_CHECK_NULL_PTR(sequence, s_tag, "GetTx: sequence is NULL");
00427   if (!state->initialized) {
00428     rx_log_error(s_tag, "GetTx: session not initialized");
00429     return k_rx_err_not_initialized;
00430   }
00431
00432   rx_err_t lock_err = internal_lock();
00433   (void)lock_err;
00434
00435   *sequence = state->tx_sequence;
00436
00437   internal_unlock();
00438
00439   /* Post-condition: *sequence <= k_session_seq_wrap_mask.
00440    * Guaranteed by the invariant on tx_sequence; no runtime check needed. */
00441
00442   return k_rx_ok;
00443 }
```

References rx_session_state_t::initialized, internal_lock(), internal_unlock(), s_tag, and rx_session_state_t::tx_sequen

Here is the call graph for this function:



4.3.2.6 rx_session_init()

```
rx_err_t rx_session_init (
            rx_session_state_t * state)   [nodiscard]
```

Initialize session state with zeroed sequences.

Sets both TX and RX sequences to k_session_initial_sequence and creates the internal ThreadX mutex for thread-safe access. Must be called before any other rx_session_*() function.

In simulator builds (RX_SIMULATOR_MODE), mutex creation is skipped since the ThreadX kernel is not running.

Precondition

state must point to valid, allocated memory

state must not already be initialized (call rx_session_deinit() first)

Postcondition

state->tx_sequence == k_session_initial_sequence

state->rx_sequence == k_session_initial_sequence

state->initialized == true

Note

Thread-safe: No mutex needed during init (single-threaded startup)

See also

rx_session_deinit() Cleanup counterpart

Since

Version 1.0.0

Definition at line 171 of file rx_session.c.

```
00172 {
00173   RX_CHECK_NULL_PTR(state, s_tag, "Init: state is NULL");
00174
00175   state->tx_sequence = k_session_initial_sequence;
00176   state->rx_sequence = k_session_initial_sequence;
00177   state->initialized = true;
00178
00179 #if !RX_IS_SIMULATOR
00180   UINT status = tx_mutex_create(&s_session_mutex, "SessionMutex", TX_NO_INHERIT);
00181   if (status != TX_SUCCESS) {
00182     rx_log_error(s_tag, "Mutex creation failed");
00183     state->initialized = false;
00184     return k_rx_err_rtos_mutex;
00185   }
00186 #endif
00187
00188   /* Post-condition: verify initialization succeeded */
00189
00190   rx_log_info(s_tag, "Session initialized (tx=0, rx=0)");
00191   return k_rx_ok;
00192 }
```

References rx_session_state_t::initialized, k_session_initial_sequence, rx_session_state_t::rx_sequence, s_session_mutex, s_tag, and rx_session_state_t::tx_sequence.

4.3.2.7 rx_session_next_tx()

```
rx_err_t rx_session_next_tx (
                rx_session_state_t * state,
                uint16_t * sequence)   [nodiscard]
```

Get next TX sequence number and increment the counter.

Atomically reads the current TX sequence, increments it (with wraparound at k_session_seq_wrap_mask), and returns the pre-increment value. This is used by both USB and SPI transport layers when sending frames.

Mirrors Go gateway's SessionState.NextTxSequence().

Precondition

state must be initialized via rx_session_init()

sequence must point to valid memory

Postcondition

state->tx_sequence incremented by 1 (with wraparound)

*sequence contains the pre-increment value

*sequence <= k_session_seq_wrap_mask

Note

Thread-safe: Protected by internal mutex

See also

rx_session_get_tx() Read without incrementing

Since

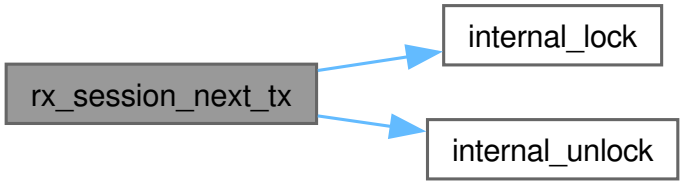
Version 1.0.0

Definition at line 260 of file rx_session.c.

```
00261 {
00262   RX_CHECK_NULL_PTR(state, s_tag, "NextTx: state is NULL");
00263   RX_CHECK_NULL_PTR(sequence, s_tag, "NextTx: sequence is NULL");
00264   if (!state->initialized) {
00265     rx_log_error(s_tag, "NextTx: session not initialized");
00266     return k_rx_err_not_initialized;
00267   }
00268
00269   rx_err_t lock_err = internal_lock();
00270   (void)lock_err;
00271
00272   *sequence          = state->tx_sequence;
00273   state->tx_sequence = (state->tx_sequence + 1) & k_session_seq_wrap_mask;
00274
00275   internal_unlock();
00276
00277   /* Post-condition: *sequence <= k_session_seq_wrap_mask.
00278    * Guaranteed by the bitwise AND above; no runtime check needed. */
00279
00280   return k_rx_ok;
00281 }
```

References rx_session_state_t::initialized, internal_lock(), internal_unlock(), k_session_seq_wrap_mask, s_tag, and rx_session_state_t::tx_sequence.

Here is the call graph for this function:



4.3.2.8 rx_session_reset()

```
rx_err_t rx_session_reset (
            rx_session_state_t * state)   [nodiscard]
```

Reset both TX and RX sequences to initial values.

Reset both TX and RX sequences to zero.

Called after a RESET/RESET_ACK handshake completes to synchronize sequences between firmware and gateway. Both counters are atomically set to k_session_initial_sequence.

Mirrors Go gateway's SessionState.Reset().

Precondition

state must be initialized via rx_session_init()

Postcondition

state->tx_sequence == k_session_initial_sequence

state->rx_sequence == k_session_initial_sequence

Note

Thread-safe: Protected by internal mutex

See also

rx_frame_type_t::k_frame_type_reset RESET frame triggers this

rx_frame_type_t::k_frame_type_reset_ack RESET_ACK confirms reset

Since

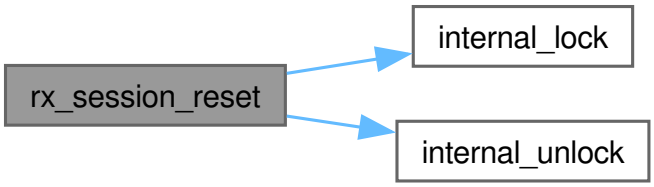
Version 1.0.0

Definition at line 384 of file rx_session.c.

```
00385 {
00386   RX_CHECK_NULL_PTR(state, s_tag, "Reset: state is NULL");
00387   if (!state->initialized) {
00388     rx_log_error(s_tag, "Reset: session not initialized");
00389     return k_rx_err_not_initialized;
00390   }
00391
00392   rx_err_t lock_err = internal_lock();
00393   (void)lock_err;
00394
00395   state->tx_sequence = k_session_initial_sequence;
00396   state->rx_sequence = k_session_initial_sequence;
00397
00398   internal_unlock();
00399
00400   rx_log_info(s_tag, "Session reset (tx=0, rx=0)");
00401   return k_rx_ok;
00402 }
```

References rx_session_state_t::initialized, internal_lock(), internal_unlock(), k_session_initial_sequence, rx_session_state_t::rx_sequence, s_tag, and rx_session_state_t::tx_sequence.

Here is the call graph for this function:



4.3.2.9 rx_session_validate_rx()

```
rx_err_t rx_session_validate_rx (
                rx_session_state_t * state,
                uint16_t received_seq,
                rx_session_validate_result_t * result)   [nodiscard]
```

Validate a received sequence number against expected RX sequence.

Validate a received sequence number.

Checks whether the received sequence number is acceptable based on the expected RX sequence. Implements gap tolerance matching the Go gateway's SessionState.ValidateRxSequence().

4.3.2.10 Validation Logic

Condition	Result	RX Sequence Update
diff == k_session_diff_exact_match	k_session_validate_ok	rx_sequence = received + 1
0 < diff < k_session_max_gap_tolerance	k_session_validate_gap	rx_sequence = received + 1

Condition	Result	RX Sequence Update
diff >= k_session_max_gap_tolerance	k_session_validate_fail	rx_sequence unchanged

Precondition

state must be initialized via rx_session_init()

Postcondition

On accept: state->rx_sequence updated to received_seq + 1

On reject: state->rx_sequence unchanged

Note

Thread-safe: Protected by internal mutex

See also

k_session_max_gap_tolerance Maximum acceptable gap

Since

Version 1.0.0

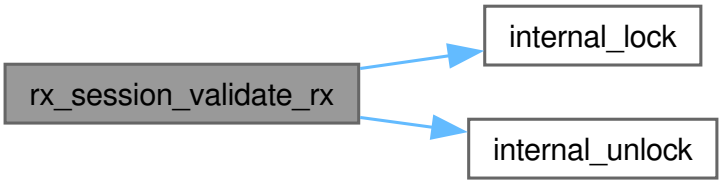
Definition at line 310 of file rx_session.c.

```
00313 {
00314   RX_CHECK_NULL_PTR(state, s_tag, "ValidateRx: state is NULL");
00315   if (!state->initialized) {
00316     rx_log_error(s_tag, "ValidateRx: session not initialized");
00317     return k_rx_err_not_initialized;
00318   }
00319
00320   rx_err_t lock_err = internal_lock();
00321   (void)lock_err;
00322
00323   /* Calculate difference using unsigned 16-bit arithmetic (handles wraparound) */
00324   uint16_t diff = received_seq - state->rx_sequence;
00325
00326   /* Exact match - most common case */
00327   if (diff == k_session_diff_exact_match) {
00328     state->rx_sequence = (state->rx_sequence + 1) & k_session_seq_wrap_mask;
00329     internal_unlock();
00330
00331     if (result != NULL) {
00332       *result = k_session_validate_ok;
00333     }
00334     return k_rx_ok;
00335   }
00336
00337   /* Small gap (packet loss) - accept and catch up.
00338    * diff > k_session_diff_exact_match is always true here: the exact-match case
00339    * already returned above, so the condition reduces to a single comparison. */
00340   if (diff < k_session_max_gap_tolerance) {
00341     rx_log_warn_val(s_tag, "Sequence gap detected, frames lost", diff);
00342     state->rx_sequence = (received_seq + 1) & k_session_seq_wrap_mask;
00343     internal_unlock();
00344
00345     if (result != NULL) {
00346       *result = k_session_validate_gap;
00347     }
00348     return k_rx_ok;
00349   }
00350
00351   /* Large gap or duplicate (diff >= gap tolerance or near-65535 wraparound) */
00352   rx_log_error_val(s_tag, "Sequence rejected", received_seq);
```

```
00353
00354   internal_unlock();
00355
00356   if (result != NULL) {
00357     *result = k_session_validate_fail;
00358   }
00359   return k_rx_err_protocol_error;
00360 }
```

References rx_session_state_t::initialized, internal_lock(), internal_unlock(), k_session_diff_exact_match, k_session_max_gap_tolerance, k_session_seq_wrap_mask, k_session_validate_fail, k_session_validate_gap, k_session_validate_ok, rx_session_state_t::rx_sequence, and s_tag.

Here is the call graph for this function:



4.3.3 Variable Documentation

4.3.3.1 s_session_mutex

TX_MUTEX s_session_mutex [static]

ThreadX mutex for protecting session state access.

Single mutex shared across all session operations. Created during rx_session_init() and deleted during rx_session_deinit().

Warning

Do not access directly. Use internal_lock() / internal_unlock().

Since

Version 1.0.0

Definition at line 91 of file rx_session.c.

Referenced by internal_lock(), internal_unlock(), rx_session_deinit(), and rx_session_init().

4.3.3.2 s_tag

const char s_tag[] = "SESSION" [static]

Logging tag for session state messages.

Note

Used with rx_log_*() macros for consistent log filtering

Since

Version 1.0.0

Definition at line 73 of file rx_session.c.

Referenced by internal_lock(), rx_session_deinit(), rx_session_get_rx(), rx_session_get_tx(), rx_session_init(), rx_session_next_tx(), rx_session_reset(), and rx_session_validate_rx().

4.4 rx_session.c

Go to the documentation of this file.

```
00001 /**
00002  * @file rx_session.c
00003  *
00004  * @brief Shared session state implementation for cross-transport sequence continuity
00005  *
00006  * @details
00007  * Implements the cross-transport shared session state module. Provides
00008  * thread-safe TX/RX sequence tracking with gap tolerance, mirroring the
00009  * Go gateway's `manager/session.go` implementation.
00010  *
00011  * Both USB CDC and SPI transports share a single session state instance
00012  * (owned by rx_comm_manager_t), ensuring sequence continuity when the active
00013  * transport switches.
00014  *
00015  * ## Thread Safety
00016  *
00017  * All public functions acquire the internal ThreadX mutex before accessing
00018  * state. The mutex uses TX_NO_INHERIT (no priority inheritance) since
00019  * contention is expected to be low (comm_task is the primary consumer).
00020  *
00021  * In simulator builds (RX_SIMULATOR_MODE), mutex operations are skipped
00022  * since the ThreadX kernel is not running.
00023  *
00024  * @pre Caller must call rx_session_init() before any other API function.
00025  * @post After rx_session_deinit(), no API calls may be made until re-init.
00026  *
00027  * @par NASA Power of 10 Compliance:
00028  * - Rule 1: No goto/setjmp/longjmp/recursion
00029  * - Rule 5: RX_CHECK_NULL_PTR / RX_VALIDATE_INIT preconditions on all functions
00030  * - Rule 7: All return values checked (RX_RETURN_ON_ERROR)
00031  * - Rule 8: Macros only for conditional compilation and validation
00032  *
00033  * @par SOLID Compliance:
00034  * - S: Single responsibility (sequence tracking only)
00035  * - D: Depends on rx_err_t abstraction, not concrete transport details
00036  *
00037  * @author Locked, Inc. Team
00038  * @date February 2026
00039  * @version 1.0.0
00040  * @copyright Copyright (c) 2026 Locked Inc.
00041  * SPDX-License-Identifier: MIT
00042  *
00043  * @see rx_session.h  Public API
00044  * @see star-gateway/internal/manager/session.go  Go reference implementation
00045  *
00046  * @since Version 1.0.0
00047  */
00048
00049 /*
      ============================================================================
00050  * Includes
00051  *
      ============================================================================
      */
00052
00053 #include "rx_session.h"
00054
00055 #include "rx_check.h"
00056 #include "rx_log.h"
00057 #include "rx_simulator_config.h"
00058
00059 #if !RX_IS_SIMULATOR
00060 #include "tx_api.h"
00061 #endif
00062
00063 /*
      ============================================================================
00064  * Constants
00065  *
      ============================================================================
      */
00066
00067 /**
00068  * @var s_tag
00069  * @brief Logging tag for session state messages
00070  * @note Used with rx_log_*() macros for consistent log filtering
00071  * @since Version 1.0.0
00072  */
00073 static const char s_tag[] = "SESSION";
00074
00075 /*
      ============================================================================
```

```
00076  * Private Data
00077  *
       ============================================================================
       */
00078
00079 #if !RX_IS_SIMULATOR
00080 /**
00081  * @var s_session_mutex
00082  * @brief ThreadX mutex for protecting session state access
00083  *
00084  * @details
00085  * Single mutex shared across all session operations. Created during
00086  * rx_session_init() and deleted during rx_session_deinit().
00087  *
00088  * @warning Do not access directly. Use internal_lock() / internal_unlock().
00089  * @since Version 1.0.0
00090  */
00091 static TX_MUTEX s_session_mutex;
00092 #endif
00093
00094 /*
       ============================================================================
00095  * Private Functions
00096  *
       ============================================================================
       */
00097
00098 /**
00099  * @brief Acquire the session mutex
00100  *
00101  * @details
00102  * Locks the internal mutex with TX_WAIT_FOREVER. In simulator mode, this
00103  * is a no-op since ThreadX is not running.
00104  *
00105  *
00106  * @pre Mutex must have been created via rx_session_init()
00107  * @post Mutex is held by calling thread
00108  *
00109  * @note Thread-safe: This IS the synchronization primitive
00110  * @since Version 1.0.0
00111  */
00112 static rx_err_t internal_lock(void)
00113 {
00114 #if !RX_IS_SIMULATOR
00115   UINT status = tx_mutex_get(&s_session_mutex, TX_WAIT_FOREVER);
00116   if (status != TX_SUCCESS) {
00117     rx_log_error(s_tag, "Mutex lock failed");
00118     return k_rx_err_rtos_mutex;
00119   }
00120 #endif
00121   return k_rx_ok;
00122 }
00123
00124 /**
00125  * @brief Release the session mutex
00126  *
00127  * @details
00128  * Unlocks the internal mutex. In simulator mode, this is a no-op.
00129  *
00130  * @pre Mutex must be held by calling thread
00131  * @post Mutex is released
00132  *
00133  * @note Thread-safe: This IS the synchronization primitive
00134  * @since Version 1.0.0
00135  */
00136 static void internal_unlock(void)
00137 {
00138 #if !RX_IS_SIMULATOR
00139   (void)tx_mutex_put(&s_session_mutex);
00140 #endif
00141 }
00142
00143 /*
       ============================================================================
00144  * Public Functions
00145  *
       ============================================================================
       */
00146
00147 /**
00148  * @brief Initialize session state with zeroed sequences
00149  *
00150  * @details
00151  * Sets both TX and RX sequences to k_session_initial_sequence and creates
00152  * the internal ThreadX mutex for thread-safe access. Must be called before
00153  * any other rx_session_*() function.
00154  *
```

```
00155  * In simulator builds (RX_SIMULATOR_MODE), mutex creation is skipped since
00156  * the ThreadX kernel is not running.
00157  *
00158  *
00159  *
00160  * @pre state must point to valid, allocated memory
00161  * @pre state must not already be initialized (call rx_session_deinit() first)
00162  * @post state->tx_sequence == k_session_initial_sequence
00163  * @post state->rx_sequence == k_session_initial_sequence
00164  * @post state->initialized == true
00165  *
00166  * @note Thread-safe: No mutex needed during init (single-threaded startup)
00167  *
00168  * @see rx_session_deinit() Cleanup counterpart
00169  * @since Version 1.0.0
00170  */
00171 rx_err_t rx_session_init(rx_session_state_t* state)
00172 {
00173   RX_CHECK_NULL_PTR(state, s_tag, "Init: state is NULL");
00174
00175   state->tx_sequence = k_session_initial_sequence;
00176   state->rx_sequence = k_session_initial_sequence;
00177   state->initialized = true;
00178
00179 #if !RX_IS_SIMULATOR
00180   UINT status = tx_mutex_create(&s_session_mutex, "SessionMutex", TX_NO_INHERIT);
00181   if (status != TX_SUCCESS) {
00182     rx_log_error(s_tag, "Mutex creation failed");
00183     state->initialized = false;
00184     return k_rx_err_rtos_mutex;
00185   }
00186 #endif
00187
00188   /* Post-condition: verify initialization succeeded */
00189
00190   rx_log_info(s_tag, "Session initialized (tx=0, rx=0)");
00191   return k_rx_ok;
00192 }
00193
00194 /**
00195  * @brief Deinitialize session state and release resources
00196  *
00197  * @details
00198  * Deletes the internal ThreadX mutex and marks the session as uninitialized.
00199  * After this call, no other rx_session_*() function may be called until
00200  * rx_session_init() is called again.
00201  *
00202  * In simulator builds, mutex deletion is skipped since it was never created.
00203  *
00204  *
00205  *
00206  * @pre state !=nullptr
00207  * @pre state->initialized == true
00208  * @post state->initialized == false
00209  * @post Internal mutex deleted (on non-simulator builds)
00210  *
00211  * @note Thread-safe: Caller must ensure no other threads are using the session
00212  *
00213  * @see rx_session_init() Initialization counterpart
00214  * @since Version 1.0.0
00215  */
00216 rx_err_t rx_session_deinit(rx_session_state_t* state)
00217 {
00218   RX_CHECK_NULL_PTR(state, s_tag, "Deinit: state is NULL");
00219   if (!state->initialized) {
00220     rx_log_error(s_tag, "Deinit: session not initialized");
00221     return k_rx_err_not_initialized;
00222   }
00223
00224 #if !RX_IS_SIMULATOR
00225   /* tx_mutex_delete return value deliberately discarded: mutex deletion errors
00226    * are non-fatal during deinit because the mutex may already be uninitialized
00227    * or no recovery is possible. The module is being torn down regardless. */
00228   (void)tx_mutex_delete(&s_session_mutex);
00229 #endif
00230
00231   state->initialized = false;
00232
00233   rx_log_info(s_tag, "Session deinitialized");
00234   return k_rx_ok;
00235 }
00236
00237 /**
00238  * @brief Get next TX sequence number and increment the counter
00239  *
00240  * @details
00241  * Atomically reads the current TX sequence, increments it (with wraparound
```

```
00242  * at k_session_seq_wrap_mask), and returns the pre-increment value. This is
00243  * used by both USB and SPI transport layers when sending frames.
00244  *
00245  * Mirrors Go gateway's `SessionState.NextTxSequence()`.
00246  *
00247  *
00248  *
00249  * @pre state must be initialized via rx_session_init()
00250  * @pre sequence must point to valid memory
00251  * @post state->tx_sequence incremented by 1 (with wraparound)
00252  * @post *sequence contains the pre-increment value
00253  * @post *sequence <= k_session_seq_wrap_mask
00254  *
00255  * @note Thread-safe: Protected by internal mutex
00256  *
00257  * @see rx_session_get_tx() Read without incrementing
00258  * @since Version 1.0.0
00259  */
00260 rx_err_t rx_session_next_tx(rx_session_state_t* state, uint16_t* sequence)
00261 {
00262   RX_CHECK_NULL_PTR(state, s_tag, "NextTx: state is NULL");
00263   RX_CHECK_NULL_PTR(sequence, s_tag, "NextTx: sequence is NULL");
00264   if (!state->initialized) {
00265     rx_log_error(s_tag, "NextTx: session not initialized");
00266     return k_rx_err_not_initialized;
00267   }
00268
00269   rx_err_t lock_err = internal_lock();
00270   (void)lock_err;
00271
00272   *sequence          = state->tx_sequence;
00273   state->tx_sequence = (state->tx_sequence + 1) & k_session_seq_wrap_mask;
00274
00275   internal_unlock();
00276
00277   /* Post-condition: *sequence <= k_session_seq_wrap_mask.
00278    * Guaranteed by the bitwise AND above; no runtime check needed. */
00279
00280   return k_rx_ok;
00281 }
00282
00283 /**
00284  * @brief Validate a received sequence number against expected RX sequence
00285  *
00286  * @details
00287  * Checks whether the received sequence number is acceptable based on the
00288  * expected RX sequence. Implements gap tolerance matching the Go gateway's
00289  * `SessionState.ValidateRxSequence()`.
00290  *
00291  * ## Validation Logic
00292  *
00293  * | Condition | Result | RX Sequence Update |
00294  * |---|---|---|
00295  * | diff == k_session_diff_exact_match | k_session_validate_ok | rx_sequence = received + 1 |
00296  * | 0 < diff < k_session_max_gap_tolerance | k_session_validate_gap | rx_sequence = received + 1 |
00297  * | diff >= k_session_max_gap_tolerance | k_session_validate_fail | rx_sequence unchanged |
00298  *
00299  *
00300  *
00301  * @pre state must be initialized via rx_session_init()
00302  * @post On accept: state->rx_sequence updated to received_seq + 1
00303  * @post On reject: state->rx_sequence unchanged
00304  *
00305  * @note Thread-safe: Protected by internal mutex
00306  *
00307  * @see k_session_max_gap_tolerance Maximum acceptable gap
00308  * @since Version 1.0.0
00309  */
00310 rx_err_t rx_session_validate_rx(rx_session_state_t*          state,
00311                                 uint16_t                    received_seq,
00312                                 rx_session_validate_result_t* result)
00313 {
00314   RX_CHECK_NULL_PTR(state, s_tag, "ValidateRx: state is NULL");
00315   if (!state->initialized) {
00316     rx_log_error(s_tag, "ValidateRx: session not initialized");
00317     return k_rx_err_not_initialized;
00318   }
00319
00320   rx_err_t lock_err = internal_lock();
00321   (void)lock_err;
00322
00323   /* Calculate difference using unsigned 16-bit arithmetic (handles wraparound) */
00324   uint16_t diff = received_seq - state->rx_sequence;
00325
00326   /* Exact match - most common case */
00327   if (diff == k_session_diff_exact_match) {
00328     state->rx_sequence = (state->rx_sequence + 1) & k_session_seq_wrap_mask;
```

```
00329     internal_unlock();
00330
00331     if (result != NULL) {
00332       *result = k_session_validate_ok;
00333     }
00334     return k_rx_ok;
00335   }
00336
00337   /* Small gap (packet loss) - accept and catch up.
00338    * diff > k_session_diff_exact_match is always true here: the exact-match case
00339    * already returned above, so the condition reduces to a single comparison. */
00340   if (diff < k_session_max_gap_tolerance) {
00341     rx_log_warn_val(s_tag, "Sequence gap detected, frames lost", diff);
00342     state->rx_sequence = (received_seq + 1) & k_session_seq_wrap_mask;
00343     internal_unlock();
00344
00345     if (result != NULL) {
00346       *result = k_session_validate_gap;
00347     }
00348     return k_rx_ok;
00349   }
00350
00351   /* Large gap or duplicate (diff >= gap tolerance or near-65535 wraparound) */
00352   rx_log_error_val(s_tag, "Sequence rejected", received_seq);
00353
00354   internal_unlock();
00355
00356   if (result != NULL) {
00357     *result = k_session_validate_fail;
00358   }
00359   return k_rx_err_protocol_error;
00360 }
00361
00362 /**
00363  * @brief Reset both TX and RX sequences to initial values
00364  *
00365  * @details
00366  * Called after a RESET/RESET_ACK handshake completes to synchronize sequences
00367  * between firmware and gateway. Both counters are atomically set to
00368  * k_session_initial_sequence.
00369  *
00370  * Mirrors Go gateway's `SessionState.Reset()`.
00371  *
00372  *
00373  *
00374  * @pre state must be initialized via rx_session_init()
00375  * @post state->tx_sequence == k_session_initial_sequence
00376  * @post state->rx_sequence == k_session_initial_sequence
00377  *
00378  * @note Thread-safe: Protected by internal mutex
00379  *
00380  * @see rx_frame_type_t::k_frame_type_reset RESET frame triggers this
00381  * @see rx_frame_type_t::k_frame_type_reset_ack RESET_ACK confirms reset
00382  * @since Version 1.0.0
00383  */
00384 rx_err_t rx_session_reset(rx_session_state_t* state)
00385 {
00386   RX_CHECK_NULL_PTR(state, s_tag, "Reset: state is NULL");
00387   if (!state->initialized) {
00388     rx_log_error(s_tag, "Reset: session not initialized");
00389     return k_rx_err_not_initialized;
00390   }
00391
00392   rx_err_t lock_err = internal_lock();
00393   (void)lock_err;
00394
00395   state->tx_sequence = k_session_initial_sequence;
00396   state->rx_sequence = k_session_initial_sequence;
00397
00398   internal_unlock();
00399
00400   rx_log_info(s_tag, "Session reset (tx=0, rx=0)");
00401   return k_rx_ok;
00402 }
00403
00404 /**
00405  * @brief Get current TX sequence without incrementing
00406  *
00407  * @details
00408  * Returns the current TX sequence counter value for diagnostic purposes.
00409  * Does not modify the counter.
00410  *
00411  *
00412  *
00413  * @pre state must be initialized via rx_session_init()
00414  * @pre sequence must point to valid memory
00415  * @post state unchanged
```

```
00416  * @post *sequence <= k_session_seq_wrap_mask
00417  *
00418  * @note Thread-safe: Protected by internal mutex
00419  *
00420  * @see rx_session_next_tx() Read and increment
00421  * @since Version 1.0.0
00422  */
00423 rx_err_t rx_session_get_tx(const rx_session_state_t* state, uint16_t* sequence)
00424 {
00425   RX_CHECK_NULL_PTR(state, s_tag, "GetTx: state is NULL");
00426   RX_CHECK_NULL_PTR(sequence, s_tag, "GetTx: sequence is NULL");
00427   if (!state->initialized) {
00428     rx_log_error(s_tag, "GetTx: session not initialized");
00429     return k_rx_err_not_initialized;
00430   }
00431
00432   rx_err_t lock_err = internal_lock();
00433   (void)lock_err;
00434
00435   *sequence = state->tx_sequence;
00436
00437   internal_unlock();
00438
00439   /* Post-condition: *sequence <= k_session_seq_wrap_mask.
00440    * Guaranteed by the invariant on tx_sequence; no runtime check needed. */
00441
00442   return k_rx_ok;
00443 }
00444
00445 /**
00446  * @brief Get current expected RX sequence without modifying
00447  *
00448  * @details
00449  * Returns the next expected RX sequence counter value for diagnostic purposes.
00450  * Does not modify the counter.
00451  *
00452  *
00453  *
00454  * @pre state must be initialized via rx_session_init()
00455  * @pre sequence must point to valid memory
00456  * @post state unchanged
00457  * @post *sequence <= k_session_seq_wrap_mask
00458  *
00459  * @note Thread-safe: Protected by internal mutex
00460  *
00461  * @see rx_session_validate_rx() Validate and update
00462  * @since Version 1.0.0
00463  */
00464 rx_err_t rx_session_get_rx(const rx_session_state_t* state, uint16_t* sequence)
00465 {
00466   RX_CHECK_NULL_PTR(state, s_tag, "GetRx: state is NULL");
00467   RX_CHECK_NULL_PTR(sequence, s_tag, "GetRx: sequence is NULL");
00468   if (!state->initialized) {
00469     rx_log_error(s_tag, "GetRx: session not initialized");
00470     return k_rx_err_not_initialized;
00471   }
00472
00473   rx_err_t lock_err = internal_lock();
00474   (void)lock_err;
00475
00476   *sequence = state->rx_sequence;
00477
00478   internal_unlock();
00479
00480   /* Post-condition: *sequence <= k_session_seq_wrap_mask.
00481    * Guaranteed by the invariant on rx_sequence; no runtime check needed. */
00482
00483   return k_rx_ok;
00484 }
```